XRPL-Standards Proposal draft, pre-number (rendered from `XLS-CollateralLock.md`). Format: XLS_TEMPLATE + AMENDMENT_TEMPLATE from XRPLF/XRPL-Standards. Reference implementation: rippled branch `collateral-lock` @ `5df6bfbc9` on tag 3.4.1.

title	Amendment XLS: Collateral Lock for Loans
description	Ties an XLS-66 loan to a token escrow so the ledger refuses to release the collateral until the loan is repaid.
implementation	(not yet public; reference branch described in section 7)
author	Harry Malinski (LiquidAcre) <harrym@liquidacre.com>
category	Amendment
status	Proposal
proposal-from	(to be opened as an XRPL-Standards Proposal discussion)
requires	XLS-33, XLS-65, XLS-66, XLS-85
created	2026-10-24
updated	2026-10-24

Amendment XLS: Collateral Lock for Loans

> This is a pre-number **Proposal**. The `xls` number is assigned by the XLS Editors and is deliberately absent. The status value follows XLS-0001's Proposal stage; it becomes Draft, with a number, when the Editors accept the Proposal. The field code and result code values below are the ones used by the reference implementation and are offered for the maintainers to confirm.

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY" and "OPTIONAL" in this document are to be interpreted as described in BCP 14 (RFC 2119 and RFC 8174) when, and only when, they appear in all capitals.

1. Abstract

The **CollateralLock** amendment lets an XLS-66 `Loan` pledge an existing XLS-85 token `Escrow` as collateral.

- **Pledging:** `LoanSet` takes an optional `CollateralID` naming an escrow owned by, and payable to, the borrower. The loan records the escrow's ID, and the escrow records the loan's ID in its existing `LoanID` field.
- **While the loan is unpaid:** `EscrowFinish` and `EscrowCancel` on that escrow fail with a new result code, `tecPLEDGED`, for every account.
- **When the loan is paid off:** the normal escrow rules apply again.
- **When the loan defaults:** the same `LoanManage` transaction moves the escrowed amount to the loan broker's owner and deletes the escrow.

The design adds one optional field to `Loan` and `LoanSet`, reuses one existing field on `Escrow`, and adds one result code and one invariant. It uses no oracle and no liquidation engine, and a loan without `CollateralID` behaves exactly as today.

2. Motivation

XLS-66 deliberately omits on-ledger collateral: a loan's only on-ledger security is the broker's first-loss cover. Builders who need collateral today pair a loan with an escrow that does not know the loan exists. Release then depends on a trusted party:

- a key service holding a crypto-condition secret;
- a creditor revealing a preimage;
- a monitoring bot calling `EscrowFinish` after `LoanManage`;
- or a timer (`FinishAfter` / `CancelAfter`).

This produces two failures that no application can fully prevent:

1. **The lender's collateral can walk.** Once `CancelAfter` passes, the borrower can cancel a pledged escrow while the loan is still unpaid. A party holding the release condition can finish it early.
2. **The borrower's collateral can be taken after repayment.** With a time-based escrow to the lender's agent, the agent can still finish the escrow after `FinishAfter` even when the loan was repaid, and the borrower recovers it only after `CancelAfter`.

Both were reproduced on XRPL Devnet before this proposal (section 6.4). CollateralLock makes the pledge a ledger fact: the release rule becomes the loan's own state.

3. Specification

3.1. Terminology

- **Pledged escrow:** an `Escrow` whose `LoanID` field names a `Loan`, set by a `LoanSet` with `CollateralID`.
- **Unpaid loan:** a `Loan` with `PaymentRemaining > 0`, or with the `lsfLoanDefault` flag set.
- **Paid off:** a `Loan` with `PaymentRemaining == 0` and without `lsfLoanDefault`, or a `Loan` that no longer exists.
- **Broker owner:** the `Owner` of the `LoanBroker` that issued the loan.
- **Maturity:** `StartDate + PaymentInterval × PaymentTotal + GracePeriod` of the loan being created. Fields absent from the `LoanSet` take the protocol defaults (`kDefaultPaymentInterval`, `kDefaultPaymentTotal`, `kDefaultGracePeriod`); `StartDate` is the loan start date as computed by `getStartDate`.

3.2. Amendment

- Name: `CollateralLock`. `Supported::Yes` and `VoteBehavior::DefaultNo` in the reference implementation, which is the setting for testing on a standalone node. An upstream pull request SHOULD start as `Supported::No` until review.

- Functionally depends on `SingleAssetVault`, `LendingProtocol` and `TokenEscrow` (and `MPTokensV1` for MPT collateral). The amendment does not encode these dependencies; the reference node ran with those five plus `fixTokenEscrowV1` enabled.

3.3. Result code

Code	Value	Message
`tecPLEDGED`	201 (proposed; after `tecNO_SPONSOR_PERMISSION` = 200 at the 3.4.1 tag; develop already uses 201 and 202, so the rebase takes the next free code)	"The escrow is pledged as collateral for an unpaid loan."

A dedicated code is used so that wallets and explorers can tell "this escrow backs an unpaid loan" apart from the existing meanings of `tecNO_PERMISSION` and `tecNO_TARGET` on escrow transactions (see also A.2).

3.4. Serialized field

Field	Type	Type code	Field code	Notes
`CollateralID`	`UINT256`	5	43 (proposed)	New. Free at the 3.4.1 tag. At the time of writing 42 was reported as taken by `ProposalID` on `develop` and 43 as free; both are to be re-confirmed at rebase.
`LoanID`	`UINT256`	5	38 (existing)	Existing field, reused on `Escrow`.

3.5. Ledger Entry: `Loan` (modified)

3.5.1. Fields

Field Name	Constant	Required	Internal Type	Default Value	Description
`CollateralID`	Yes	No	`UINT256`	N/A	The key of the `Escrow` pledged as collateral for this loan. Set only by `LoanSet`.

All other `Loan` fields are unchanged. Flags, ownership, reserves, deletion, RPC name and pseudo-account are unchanged from XLS-66.

3.5.2. Invariants

- `CollateralID` MUST NOT change after the loan is created. The reference implementation adds it to `NoModifiedUnmodifiableFields`.
- A `Loan` created with `CollateralID` MUST name an `Escrow` whose `LoanID` names that `Loan` (`ValidCollateralLock`, section 3.11).

3.5.3. Example JSON

From the reference node's proof run 3 (ledger 64; build `5df6bfbc9`):

```
{
  "LedgerEntryType": "Loan",
  "LedgerIndex": "04F12C107D8AEFCE4C078B123D10FED99CCC3752F63863ED8A1AA4CE4BCADA23",
  "Borrower": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "LoanBrokerID": "1218BE4C5C7A5526D1B8F664B37B874FA2B7796F2A145997085751303 20DAF08",
  "LoanSequence": 1,
  "CollateralID": "9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9",
  "PrincipalOutstanding": "1000000000",
  "TotalValueOutstanding": "1000000000",
  "PeriodicPayment": "333333333.3333333333",
  "PaymentInterval": 60,
  "PaymentRemaining": 3,
  "GracePeriod": 60,
  "StartDate": 846208790,
  "NextPaymentDueDate": 846208850
}
```


3.6. Ledger Entry: `Escrow` (modified)

3.6.1. Fields

Field Name	Constant	Required	Internal Type	Default Value	Description
`LoanID`	Yes, once set	No	`UINT256`	N/A	The key of the `Loan` this escrow is pledged to. Set only by `LoanSet` with `CollateralID`. Never changed or removed while the escrow exists.

Flags, ownership and RPC name are unchanged.

3.6.2. Deletion

A pledged escrow MAY be deleted only:

- by `EscrowFinish` or `EscrowCancel` once its loan is paid off or no longer exists (normal escrow rules then apply), or
- by the `LoanManage` transaction with `tfLoanDefault` that sweeps it (section 3.10).

3.6.3. Reserves

Unchanged. The escrow already counts toward the borrower's owner reserve. On a default sweep, the borrower's owner count decreases by one.

3.6.4. Freeze and lock

- Pledging and sweeping respect IOU freeze and deep freeze and MPT lock: `LoanSet` and the default sweep both check that neither the borrower nor the broker owner is frozen (`tecFROZEN`, IOU) or locked (`tecLOCKED`, MPT) for the asset (section 3.7.2).

- If the issuer freezes or locks the collateral mid-loan, `EscrowFinish` is blocked as today and the escrow stays pledged.
- If the broker owner can no longer receive the asset at default time, the default fails with that code and the escrow stays pledged (section 8).

3.6.5. Invariants

See section 3.11.

3.6.6. Example JSON

The escrow pledged by the loan above:

```
{
  "LedgerEntryType": "Escrow",
  "LedgerIndex": "9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9",
  "Account": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "Destination": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "Amount": { "mpt_issuance_id": "00000034DCE0A99842C506EF14B0F06F80B85A3E15C9D6CA", "value": "1
0000000" },
  "FinishAfter": 846208778,
  "CancelAfter": 846212376,
  "LoanID": "04F12C107D8AEFCE4C078B123D10FED99CCC3752F63863ED8A1AA4CE4BCADA23",
  "Flags": 0,
  "OwnerNode": "0"
}
```

3.7. Transaction: `LoanSet` (modified)

3.7.1. Fields

Field Name	Required?	JSON Type	Internal Type	Default Value	Description
`CollateralID`	No	`string` (hex)	`UINT256`	N/A	The key of the borrower's token `Escrow` to pledge as collateral.

Flags and transaction fee are unchanged.

3.7.2. Failure Conditions

3.7.2.1. Data Verification

1. `CollateralID` is present and `CollateralLock` is not enabled: `temDISABLED`.
2. `CollateralID` is zero: `temINVALID`.

3.7.2.2. Protocol-Level Failures

Evaluated in this order, after all existing XLS-66 preclaim checks:

1. No `Escrow` exists at `CollateralID`: `tecNO_ENTRY`.
2. The escrow's `Amount` is XRP (only IOU and MPT escrows can be pledged): `tecWRONG_ASSET`.
3. The escrow's `Account` or `Destination` is not the borrower: `tecNO_PERMISSION`.
4. The escrow already has a `LoanID`: `tecDUPLICATE`.
5. The escrow has a `CancelAfter` earlier than the loan's maturity: `tecTOO_SOON`.
6. Collateral checks against the broker owner (skipped entirely if the broker owner is the asset's issuer):
7. The broker owner holds no trust line or MPToken for the asset: `tecNO_AUTH`.
8. The broker owner is not authorised for the asset (`RequireAuth` / MPT authorisation): the code returned by `requireAuth`.
9. The borrower, then the broker owner, is frozen (IOU) or locked (MPT) for the asset: `tecFROZEN` or `tecLOCKED`.
10. The asset cannot be transferred from the borrower to the broker owner: the code returned by `canTransfer`.

No new code is introduced for `LoanSet`.

3.7.3. State Changes

On success, in addition to the XLS-66 effects:

- `Loan.CollateralID` = `CollateralID`;
- `Escrow.LoanID` = the new loan's key.

No funds move.

3.7.4. Metadata

The escrow appears as a `ModifiedNode` with `LoanID` added; the new `Loan` `CreatedNode` carries `CollateralID`.

3.7.5. Example JSON

From proof run 3 (the borrower signs on the Ledger XRP app, ClearPledge collateral-id build, in Speculos; the broker owner countersigns on the node):

```
{
  "TransactionType": "LoanSet",
  "Account": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "Counterparty": "rJrfTYaFMNJ8KWAkEAJk5ejN614MShYp1B",
  "LoanBrokerID": "1218BE4C5C7A5526D1B8F664B37B874FA2B7796F2A145997085751303 20DAF08",
  "PrincipalRequested": "1000000000",
  "PaymentInterval": 60,
  "PaymentTotal": 3,
  "GracePeriod": 60,
  "CollateralID": "9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9",
  "Fee": "100",
  "Sequence": 17,
  "CounterpartySignature": { "SigningPubKey": "…", "TxnSignature": "…" }
}
```


Result: `tesSUCCESS`, ledger 64 (`FEC09B55…7D63`). The device's review showed "Collateral ID" = `9B6002B5…96BFA9`.

3.8. Transaction: `EscrowFinish` (modified)

3.8.1. Fields

No fields are added or changed. Flags and transaction fee are unchanged.

3.8.2. Failure Conditions

3.8.2.1. Data Verification

No change.

3.8.2.2. Protocol-Level Failures

1. `CollateralLock` is enabled, and the target escrow has a `LoanID` whose `Loan` exists and is unpaid: `tecPLEDGED` (new; justification in section 3.3 and A.2).

- The check runs in preclaim, after the existing credential check and before the token-escrow preclaim checks. `FinishAfter` and `Condition` are evaluated later, in apply, so the check applies whether or not they would otherwise allow the transaction.
- It applies to every submitter: the escrow's owner, its destination, or a third party.
- Once the loan is paid off, or no longer exists, the existing XLS-85 behaviour applies unchanged.

3.8.3. State Changes

None beyond existing behaviour.

3.8.4. Example JSON

From proof run 3, while the loan is open:

```
{
  "TransactionType": "EscrowFinish",
  "Account": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "Owner": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "OfferSequence": 16,
  "Fee": "100",
  "Sequence": 18
}
```

Result: `tecPLEDGED`, ledger 66 (`A7A71A7D…F25E`). After three `LoanPay`, the same transaction (Sequence 23) returns `tesSUCCESS` at ledger 72 (`7B7FF9E8…B165`).

3.9. Transaction: `EscrowCancel` (modified)

3.9.1. Fields

No fields are added or changed. Flags and transaction fee are unchanged.

3.9.2. Failure Conditions

3.9.2.1. Data Verification

No change.

3.9.2.2. Protocol-Level Failures

1. `CollateralLock` is enabled, and the target escrow has a `LoanID` whose `Loan` exists and is unpaid: `tecPLEDGED`.

- This is the first check in `EscrowCancel` preclaim. `CancelAfter` is evaluated later, in apply, so the check applies even after `CancelAfter` has passed.
- It applies to every submitter. Once the loan is paid off, or no longer exists, existing behaviour applies.

3.9.3. State Changes

None beyond existing behaviour.

3.9.4. Example JSON

From proof run 3, while the loan is open:

```
{
  "TransactionType": "EscrowCancel",
  "Account": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "Owner": "ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ",
  "OfferSequence": 16,
  "Fee": "100",
  "Sequence": 19
}
```

Result: `tecPLEDGED`, ledger 68 (`5CDFEFA5…279D`).

3.10. Transaction: `LoanManage` (modified, `tfLoanDefault`)

3.10.1. Fields

No fields are added or changed. Transaction fee is unchanged.

3.10.2. Failure Conditions

3.10.2.1. Data Verification

No change.

3.10.2.2. Protocol-Level Failures

1. After the existing `LoanManage` checks, with `tfLoanDefault` set and the loan carrying `CollateralID`: the collateral cannot be moved to the broker owner (no holding, not authorised, frozen or locked, or not transferable). The result is the corresponding existing code, for example `tecNO_AUTH`, `tecFROZEN` or `tecLOCKED`. The default does not happen and the escrow stays pledged.

3.10.3. State Changes

When a default succeeds and the loan has `CollateralID`, in the same transaction and after the XLS-66 default accounting (first-loss cover applied, then the Vault written down):

1. The escrowed `Amount` is delivered to the broker owner with the same helper `EscrowFinish` uses (`escrowUnlockApplyHelper`), honouring the transfer rate locked in the escrow (`TransferRate`, else parity) and MPT locked-amount accounting. Unlike an `EscrowFinish` submitted by the destination, the sweep never creates a trust line or MPToken for the broker owner: the holding MUST already exist, which the preclaim check guarantees.
2. The escrow is removed from the borrower's owner directory, from the destination directory if `DestinationNode` is present (never for a self-escrow), and from the issuer's directory if `IssuerNode` is present, then deleted.
3. The borrower's owner count decreases by one.

A loan without `CollateralID` defaults exactly as today.

3.10.4. Metadata

The escrow appears as a `DeletedNode`; the broker owner's trust line or MPToken appears as a `ModifiedNode` with the increased balance.

3.10.5. Example JSON

From proof run 3 (second loan):

```
{
  "TransactionType": "LoanManage",
  "Account": "rJrfTYaFMNJ8KWAKEAJk5ejN614MShYp1B",
  "LoanID": "99803617E6FFACFBA808E04E08B9A8CC2FB97E82E906F13659D3D1ED9946C246",
  "Flags": 65536,
  "Fee": "10",
  "Sequence": 60
}
```

Result: `tesSUCCESS`, ledger 100 (`72D5D727…B268`); 5,000 g of the gold MPT moved to the broker owner and the escrow deleted.

3.11. Invariant: `ValidCollateralLock` (new)

For every successful transaction:

1. An `Escrow`'s `LoanID` MAY be added only by `LoanSet`, and MUST NOT be changed or removed while the escrow exists.
2. A pledged `Escrow` MAY be deleted only when its loan is paid off or no longer exists, or by a `LoanManage` that defaults its loan.
3. A `Loan` created with `CollateralID` MUST name an `Escrow` whose `LoanID` names that `Loan`.

Amount conservation of the sweep is covered by the existing `ValidMPTBalanceChanges`, `ValidMPTTransfer` and `ValidAmounts` invariants.

4. Rationale

- **Gate at release time, not a flag cleared at payoff.** `EscrowFinish` and `EscrowCancel` read the loan named by `Escrow.LoanID`. Payoff, `LoanDelete` after payoff and a missing loan are all handled without modifying `LoanPay` or `LoanDelete`.
- **Self-escrow only (`Account` = `Destination` = borrower).** After payoff the collateral can go nowhere but back to the borrower. This removes the "repaid but still seizable" failure in section 2.
- `CancelAfter` **must not precede maturity.** The gate already blocks `Cancel` at any time while the loan is unpaid. The check keeps a loan's term honest and prevents pledging an escrow that was designed to expire mid-loan.
- **The default pays the broker owner, not the Vault.**
- XLS-65 vaults are single-asset (`Vault.Asset`, `tecWRONG_ASSET` for other assets), and pseudo-accounts cannot receive payments, so collateral in a different asset (for example gold against an RLUSD vault) cannot be placed there.
- The XLS-66 default first applies the broker's cover. The collateral then goes to the broker owner, whose first-loss capital absorbed the loss.
- Senior-first recovery to depositors is left to a follow-up (`LoanRecover`, see Appendix A).
- **Token escrows only.** XLS-85 escrows carry issuer controls (freeze, lock, `RequireAuth`, transfer rate) that the sweep can honour precisely. XRP collateral uses the classic escrow path and is left for a follow-up.

- **No oracle, no liquidation engine.** The rule depends only on the loan's state. Valuation and liquidation policy, such as LTV-based margin calls, can be layered above it.
- **Alternatives considered:**
- *Application-level watchers:* the trusted step this removes.
- *Crypto-condition escrows:* the secret holder is trusted in both directions.
- *Smart Escrow (XLS-100):* a programmable finish condition could in principle read a `Loan`, but it is still a release predicate on one escrow: nothing on the `Loan` requires the escrow to exist, and the escrow keeps its own cancel path. CollateralLock's check does not depend on how the escrow's own conditions are expressed, so the two should compose; this has not been tested on the reference branch.

5. Backwards Compatibility

- No existing behaviour changes for loans without `CollateralID` or escrows without `LoanID`.
- `CollateralID` is optional and is rejected with `temDISABLED` before the amendment is enabled.
- As with any amendment, servers that do not support `CollateralLock` become amendment-blocked once it is enabled.
- **Sequencing with XLS-66 follow-ups:** if `LoanSet` becomes a two-step offer/accept flow (as has been proposed for a LendingProtocol follow-up), `CollateralID` SHOULD travel with the borrower's accepting step. The check is unchanged and moves one transaction later.

6. Test Plan

6.1. Unit tests

`xrpld --unittest=CollateralLock` (suite `xrpl.tx.CollateralLock`, `src/test/app/lending/CollateralLock_test.cpp`): 7 cases, 687 assertions, 0 failures.

1. **Disabled:** `CollateralID` returns `temDISABLED`; a `LoanSet` without it still succeeds.
2. **Loan without CollateralID is unchanged:** neither `Loan.CollateralID` nor `Escrow.LoanID` is written, and the escrow finishes as today.
3. **LoanSet preclaim:** no such escrow (`tecNO_ENTRY`), escrow owned by another account (`tecNO_PERMISSION`), escrow payable to another account (`tecNO_PERMISSION`), XRP escrow (`tecWRONG_ASSET`), `CancelAfter` before maturity (`tecTOO_SOON`), already pledged (`tecDUPLICATE`), broker owner holds no MPToken (`tecNO_AUTH`).
4. **Finish and Cancel return tecPLEDGED until payoff:** the loan and escrow point at each other; Finish by the borrower, the broker owner and a third party, and Cancel by the borrower, all return `tecPLEDGED`; after three `LoanPay` the same Finish succeeds.
5. **EscrowCancel succeeds after payoff and LoanDelete:** Finish returns `tecPLEDGED` while open; after payoff the loan is deleted and, once `CancelAfter` passes, Cancel succeeds.
6. **Default sweeps the collateral to the broker owner:** an early default returns `tecTOO_SOON`; after `CancelAfter` with the loan unpaid, Cancel and Finish return `tecPLEDGED`; `LoanManage` `tfLoanDefault` then deletes the escrow, credits the broker owner, lowers the borrower's `OwnerCount` by one and sets `lsfLoanDefault`; a further Finish returns `tecNO_TARGET`.
7. **Default fails, and the pledge stays, if the sweep is impossible:** with the broker owner's MPToken locked, the default returns `tecLOCKED` and the escrow remains; after unlock the default succeeds.

Coverage note: the suite pledges an MPT only. The IOU path (trust line, freeze, deep freeze, `RequireAuth`, transfer rate) is exercised through the shared helpers (`holdingExists`, `requireAuth`, `checkFrozen`, `canTransfer`, `escrowUnlockApplyHelper<Issue>`) but has no dedicated case yet. Adding one is the first test follow-up.

6.2. Regression

- `--unittest=Escrow,EscrowToken,Loan,LendingHelpers,Vault,Invariants,Feature,TER,MPToken,Batch`: 40 suites, 2,279 cases, 253,852 assertions, 0 failures.
- `xrpl_tests` (gtest, including the regenerated protocol wrappers): 934 passed (3 pre-existing disabled).

6.3. Integration (standalone node from the reference branch, run 3)

Build `5df6bfbc9` (the branch HEAD). The borrower's 11 transactions are signed by the Ledger XRP app (ClearPledge collateral-id build) in Speculos; a physical-device run is pending.

Step	Result	Ledger	Hash
`LoanSet` with `CollateralID`	`tesSUCCESS`	64	`FEC09B55…7D63`
`EscrowFinish` by the borrower while the loan is open	`tecPLEDGED`	66	`A7A71A7D…F25E`
`EscrowFinish` by the broker owner while the loan is open	`tecPLEDGED`	67	`D2249D9B…7D35`
`EscrowCancel` by the borrower while the loan is open	`tecPLEDGED`	68	`5CDFEFA5…279D`
Three `LoanPay` (final at ledger 71), then `EscrowFinish`	`tesSUCCESS`	72	`7B7FF9E8…B165`
Second loan with `CollateralID`	`tesSUCCESS`	74	`5FCC29B7…53AB`
`LoanManage tfLoanDefault`	`tesSUCCESS`; 5,000 g swept to the broker owner	100	`72D5D727…B268`

Full submit and tx JSON: `evidence/proof_run_result.json` in the proposal repository (standalone-node hashes are local to that node). Earlier runs 1 and 2 are kept as `proof_run_result_run1.json` and `proof_run_result_run2.json` for the record.

6.3b. Two-validator network

Two `xrpld` validators running the amendment (network_id 424, quorum 2: one on a home PC, one on a laptop in New York), recorded Sat Oct 24 23:42–23:45 EDT (Oct 25 03:42–03:45 UTC), ledgers 1250–1321. For all 23 transactions in the run, both nodes report the transaction in the same validated ledger with the same ledger hash (`evidence/proof_2val_crosscheck.json`, `all_agree: true`).

Step	Result	Ledger	Hash
`EscrowFinish` by the borrower while the loan is open	`tecPLEDGED`	1266	`84D39A0F…8D5A`
`EscrowFinish` by the broker owner while the loan is open	`tecPLEDGED`	1267	`A47A7BBA…AB17`
`EscrowCancel` by the borrower while the loan is open	`tecPLEDGED`	1268	`D43288A4…F20C`
`EscrowFinish` after payoff	`tesSUCCESS`	1273	`5520CCDA…2AEB`
`LoanManage tfLoanDefault`	`tesSUCCESS`	1321	`5AED6C88…12A2`

The network binary's version string reads `9ce853322`: the same source tree as `5df6bfbc9`, built before the commit messages were edited.

6.4. The gap, reproduced without the amendment (public Devnet)

- `EscrowFinish` releasing an escrow while its loan is unpaid: `tesSUCCESS` (`3B32B17240A4BD0E8D6D75DA97D813040B6124C90151E8B06EBAF9AD14D79D26`, 2026-10-19).
- A borrower `EscrowCancel` after `CancelAfter` with the loan unpaid: `tesSUCCESS` (`8E74D68A1EF3DC23DFF3FB32B827BD3855BDEF37252729E1813F20446876700B`, 2026-10-24).
- An agent `EscrowFinish` on a repaid loan's escrow after `FinishAfter`: `tesSUCCESS` (`E100E8DDDE102306067F3BF20E6EC6058E7E348B5E4E2E4A8D91FE7D27358FEB`, 2026-10-24; repaid in `F8130ED8CF8632A3E713DC0F3278A4990E25081F495A573F89A66AD7C39825B0`).

Devnet resets periodically; the submit and tx JSON for all three is kept under `evidence/devnet/` in the proposal repository.

7. Reference Implementation

Branch `collateral-lock`, cut from tag `3.4.1` (`00e640751`), six commits:

1. The amendment, `sfCollateralID`, `tecPLEDGED`, the Escrow/Loan/LoanSet formats and the regenerated protocol wrappers.
2. `EscrowFinish` and `EscrowCancel` return `tecPLEDGED` (`isPledged` and `canSweepCollateral` in `EscrowHelpers.h`).
3. `LoanSet` pledges a token escrow.
4. `LoanManage` default sweeps the collateral to the broker owner (reusing `escrowUnlockApplyHelper`).
5. The `ValidCollateralLock` invariant, and `CollateralID` added to `NoModifiedUnmodifiableFields`.
6. The jtx suite.

Size: about 383 lines of protocol and transactor code across 17 files, plus generated wrappers and tests. A rebase onto `develop`, where the lending transactors have moved on since 3.4.1, is the first follow-up. The link will be added when the branch is published.

8. Security Considerations

- **The stranded borrower (open issue).** If a loan is never paid off and the broker never submits `LoanManage tfLoanDefault`, the pledged escrow stays locked indefinitely, because the gate blocks `Cancel` regardless of `CancelAfter`. A follow-up SHOULD allow `EscrowCancel` once

`NextPaymentDueDate + GracePeriod` has passed by a further margin with no default, or allow the borrower to trigger the default sweep's counterpart. This is stated up front because it is the main liveness property reviewers should weigh.

- **Cancel and finish races.** Every release path is checked in preclaim against the loan's current state, so ordering within a ledger cannot release a pledged escrow while the loan is unpaid. Covered by invariant 2 and test 6.
- **Issuer controls.**
- Freeze, deep freeze, MPT lock and `RequireAuth` are honoured at pledge time and at sweep time.
- Under XLS-85 the issuer cannot be the source of a token escrow, and escrowed funds sit outside the holder's balance, so Clawback does not reach them; this proposal changes neither.
- An issuer action that blocks the sweep makes the default fail rather than move funds against the issuer's controls. The escrow stays pledged until the block is lifted.
- **Griefing.** Only the borrower's own self-escrow can be pledged, and only through a `LoanSet` the borrower signs, as submitter or as counterparty. A third party cannot lock someone else's escrow.
- **Depositor exposure.** On default the collateral goes to the broker owner, not to depositors. Depositors remain protected only by the XLS-66 cover. This matches XLS-66's first-loss model but is a policy choice that reviewers may wish to revisit (`LoanRecover`).
- **No oracle dependency.** No price feed is read, so oracle manipulation cannot trigger or block a release.
- **Ledger growth.** No new ledger entries are created. One optional field is added to `Loan` and one existing field is populated on `Escrow` .
- **Mixed-version networks.** As with any amendment, behaviour changes only once the amendment is enabled. Before that, `CollateralID` is rejected in preflight.
- **Interaction with Smart Escrow (untested).** Because the CollateralLock check runs in preclaim, it is expected to run before any programmable finish condition evaluated in apply. The reference branch is cut from 3.4.1, which does not include Smart Escrow, so this is a design expectation, not a tested result.

Appendix

Appendix A: FAQ

A.1: Why not let applications enforce this?

They can't. An application can only decide when it submits a transaction. It cannot stop the borrower cancelling after `CancelAfter` , or a condition holder finishing early. The trusted step is exactly what this removes.

A.2: Why a new result code instead of `tecNO_PERMISSION` ?

`tecNO_PERMISSION` already means several things on escrow transactions. A distinct `tecPLEDGED` lets wallets explain the refusal ("pledged to an unpaid loan") without parsing context.

A.3: Is this a replacement for XLS-66's first-loss cover?

No. It is opt-in per loan and sits alongside the cover. Most XLS-66 loans are expected to remain uncollateralised.

A.4: What is left for follow-ups?

- `LoanRecover` (senior-first recovery to depositors);
- a time-based release for a broker that never defaults;
- partial release and top-ups;
- several escrows per loan;
- surplus return;
- XRP collateral;
- a dedicated IOU-collateral test case.

Appendix B: Relation to the earlier team draft ("XLS-66c")

An earlier internal draft described the same idea with different identifiers. This document supersedes it and matches the reference implementation. The self-assigned "XLS-66c" label is dropped, because XLS numbers are assigned by the Editors.

	This document	**Earlier draft**
Loan field	`CollateralID`, UINT256 43	`CollateralEscrowID`, UINT256 42 (reported taken by `ProposalID` on `develop`)
Escrow back-pointer	existing `LoanID` (38), set by `LoanSet`	new `LoanID`, UINT256 43, set at `EscrowCreate` or `LoanSet`
Result while unpaid	`tecPLEDGED` (new)	`tecNO_PERMISSION`
Escrow destination	MUST be the borrower	holder
Default	to the broker owner	"the broker's account" (the broker pseudo-account cannot hold a non-vault asset)
Amendment	`CollateralLock`	`featureLendingCollateral`